运算符名称

In [1]:
```
print(3 + 2)    # add 加法
print(5 - 3)    # subtract 减法
print(4 * 2)    # multiply 乘法
print(7 / 2)    # divide 除法
print(7 // 2)   # floor divide 整除
print(7 % 2)    # modulus 取模/求余
print(2 ** 3)   # exponent 幂运算
```

```
5
2
8
3.5
3
1
8
```

判断奇数偶数

In [2]:
```
num = 15

# 判断奇数偶数 Odd / Even
if num % 2 == 0:
    print(num, "是偶数 Even")
else:
    print(num, "是奇数 Odd")

# 判断3的倍数 Divisible by 3
if num % 3 == 0:
    print(num, "是3的倍数 Divisible by 3")
else:
    print(num, "不是3的倍数 Not divisible by 3")
```

```
15 是奇数 Odd
15 是3的倍数 Divisible by 3
```

variable and data types

In [3]:
```
print("Hello Python world!")
```

```
Hello Python world!
```

In [4]:
```
message = "Hello Python world!"
print(message)
```

```
Hello Python world!
```

In [5]:
```
# 2-1
messag="I love python world!"
print(messag)
```

```
I love python world!
```

In [6]:
```
# 2-2
new_messag="Do you like python?"
print(new_messag)
```

```
Do you like python?
```

title() / upper() / lower()

In [8]:
```
name = "ada lovelace"
print(name.title())
```

```
Ada Lovelace
```

In [9]:
```
name = "ada lovelace"
print(name.upper())
print(name.lower())
print(name.title())
```

```
ADA LOVELACE
ada lovelace
Ada Lovelace
```

In [10]:
```
first_name="ada"
last_name="lovelace"
full_name=first_name + " " +last_name

message = "Hello, " + full_name.title() + "!"
print(message)
```

```
Hello, Ada Lovelace!
```

\n + \t + \n\t

In [12]:
```
print("I love python world.")
print("\tI love python world.")
print("\nI love python world.")
print("\n\tI love python world.")
```

```
I love python world.
        I love python world.

I love python world.

        I love python world.
```

In [13]:
```
print("Languages:\nPython\nC\nJavaScript")
```

```
Languages:
Python
C
JavaScript
```

strip() / rstrip() / lstrip() --去除空格

In [14]:
```
favorite_language = '    python      '
```

```
favorite_language.rstrip()
```

Out[14]: '   python'

In [15]:
```
favorite_language = '    python       '
favorite_language.lstrip()
```

Out[15]: 'python   '

In [17]:
```
favorite_language = '    python       '
favorite_language.strip()
```

Out[17]: 'python'

Excercise

In [18]:
```
# 2-3
name = 'eric'
print("Hello " + name.title() + ", would you like to learn some Python today?")
```

```
Hello Eric, would you like to learn some Python today?
```

In [19]:
```
# 2-4
name = 'Eric li'
print(name.title())
print(name.lower())
print(name.upper())
```

```
Eric Li
eric li
ERIC LI
```

In [20]:
```
# 2-5
name = "steven jobs"
quote = "Stay hungry,Stay foolish."
print(name.title() + ' once said,"' + quote + '"')
```

```
Steven Jobs once said,"Stay hungry,Stay foolish."
```

In [35]:
```
# 2-6
famous_person = "steven jobs"
message = famous_person.title() + ' once said,"Stay hungry, Stay foolish."'

print(message)
```

```
Steven Jobs once said,"Stay hungry, Stay foolish."
```

In [21]:
```
# 2-7-1
name = '    steven jobs       '
print(name)
print("\t" + name)
print("\n" + name)
print("\n\t" + name)
```

```
  steven jobs
          steven jobs

  steven jobs

          steven jobs
```

In [22]:
```
# 2-7-2
name = '   steven jobs      '
print(name)
print(name.lstrip())
print(name.rstrip())
print(name.strip())
```

```
   steven jobs
steven jobs
   steven jobs
steven jobs
```

In [23]:
```
# 2-7 Stripping Names
name = "\t\n  Steven Jobs  \t"

# 打印包含空白的名字
print(name)

# 使用三种去除空白的函数
print(name.lstrip())
print(name.rstrip())
print(name.strip())
```

```
  Steven Jobs
Steven Jobs

  Steven Jobs
Steven Jobs
```

add (+), subtract (-), multiply (*), and divide (/)

Integers

In [24]:
```
3**3
```

Out[24]: 27

In [25]:
```
2+3
```

Out[25]: 5

Floats

In [26]:
```
0.1 + 0.1
```

Out[26]: 0.2

In [27]:
```
0.2 + 0.1
```

Out[27]: 0.30000000000000004

str()

In [28]:
```
age = 23
message = "Happy " + str(age) + "rd Birthday!"
print(message)
```

```
Happy 23rd Birthday!
```

Excercise

In [29]:
```
# 2-8
print(5 + 3)
print(10 - 2)
print(4 * 2)
print(16 / 2)
```

```
8
8
8
8.0
```

In [30]:
```
# 2-9
num=9
print("My favorite number is " + str(num) + ".")
```

```
My favorite number is 9.
```

In [ ]:
```
# Why Learn Data Science?
Data is growing fast, and storage & computing costs are dropping.
Data science helps find insights and support smart decisions.
Data scientists are in high demand and well-paid.
It is widely used in many industries.

Data is everywhere, and data science helps us understand it.
It lets us find patterns, make predictions, and solve real problems.
It has high demand in many fields like business, medicine, and technology.
It provides good job opportunities and strong career growth.
It helps people and companies make smarter decisions.
```

In [ ]:
```
# What you can learn from Data Science
Python programming: Learn to code and process data with Python.
Statistics and math: Understand data rules and analyze numbers.
Data analysis and visualization: Find insights and show data with charts.
Machine learning basics: Build simple predictive models.
Communication and domain knowledge: Explain results and solve real problems
```

In [ ]:
```
# What Data Scientists Do
Collect, process and prepare raw data.
Ensure data quality and security.
Analyze and visualize data to discover insights.
```

```
Store and manage data for future use.
Work in teams to solve problems using data.
```

In [ ]:
```
# Key Differences Between Data Scientists & Data Engineers
1. Data scientists analyze data, while data engineers build data systems.
2. Data scientists create models, while data engineers build data pipelines.
3. Data scientists find insights, while data engineers ensure data quality.
4. Data scientists visualize data, while data engineers manage databases.
```

In [ ]:
```
# Key Term's Explanation
for loop: Repeats code for each item in a list or group.
while loop: Repeats code as long as a condition is true.
if statement: Runs different code based on true or false conditions.
function: A reusable block of code for a specific task.
list: An ordered collection of multiple items.
dictionary: Stores data as key-value pairs for organized information.
```

In [ ]:
```
# What is data science, and what disciplines does it integrate?
Data science is an interdisciplinary field that combines:
Math & Theory (statistics, linear algebra, optimization)
Applied Algorithms (machine learning, data structures)
Engineering & Technologies (storage/computing platforms, tools)
Domain Expertise (finance, text, images, econometrics)
Art (visualization, infographics)
Best Practices (data cleaning, missing value handling)
```

In [ ]:
```
# Why is data science a multi-disciplinary field, and why are all these pillars
Data science requires a full skill set:
Math/theory provides the foundational logic for analysis.
Algorithms turn theory into actionable solutions.
Engineering enables scalable big data processing.
Domain expertise ensures insights are business-relevant.
Visualization communicates results to stakeholders.
Best practices ensure data quality and reliable analysis.
No single pillar is sufficient; all are required for end-to-end data work.
```

In [ ]:
```
# What core skills does a data scientist need to master, and how do they work to
A data scientist needs 6 core skills:
Strong mathematical/statistical foundation
Proficiency in algorithms and programming
Engineering skills for data infrastructure
Deep domain knowledge of the industry
Visualization and communication skills
Practical data handling experience
These skills work together to turn raw data into actionable, business-aligned in
```

In [ ]:
```
# Why Choose Python
Simple, clean, and easy to read and write.
Shorter code than many other languages.
Used for games, websites, data science, and many real jobs.
Has a large, friendly community to help beginners.
```

In [64]:
```
import this
```

```
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

In [ ]: